Trappole d'esame — Dominio 3: Configurazione e workflow di Claude Code

> **Come usare questo file.** Per ogni trappola: risposta sbagliata (❌), perché, risposta corretta (✅), segnale nella domanda (🔎). ⚠️ **Guida vs docs** segnala dove la guida d'esame e la documentazione attuale (ottobre 2026) divergono: all'esame vale la guida.

Regole d'oro del dominio

1. **Condiviso col team = dentro il repo** (`.claude/…`, `CLAUDE.md` di progetto). **Personale = `~/.claude/…`.**
2. **Sempre attivo → `CLAUDE.md` / `.claude/rules/`; su richiesta → skill.**
3. **Convenzioni sparse in molte cartelle → regole per percorso con glob**, non un `CLAUDE.md` per cartella.
4. **Plan mode per cambi architetturali multi-file; esecuzione diretta per fix chiari e localizzati.**
5. **Trasformazione nota interpretata male → esempi input/output concreti.**
6. **CI: `claude -p`; revisione con istanza indipendente; Batch API solo per lavori non bloccanti.**

---

1. Gerarchia, ambito e organizzazione di CLAUDE.md

Posizione	Ambito	Condiviso via git?
`~/.claude/CLAUDE.md`	utente, tutti i progetti	❌
`CLAUDE.md` o `.claude/CLAUDE.md` nella radice	progetto	✅
`sottocartella/CLAUDE.md`	quella cartella	✅
`.claude/rules/*.md` con `paths:` (glob)	file che corrispondono al pattern	✅

Trappola 1 — Il nuovo collega non riceve le istruzioni pur lavorando sullo stesso repo e branch

Causa: le istruzioni stanno in `~/.claude/CLAUDE.md` (livello utente), che non è versionato.

✅ Spostale nel `CLAUDE.md` **di progetto** (es. `.claude/CLAUDE.md`), così arrivano a tutti con git.

🔎 "Stesso repo, stesso branch, comportamento diverso" → quasi sempre configurazione a livello utente.

Trappola 2 — Pensare che `/memory` attivi o ricarichi la configurazione

Perché è sbagliato: nessun comando "attiva" i file: si caricano in base alla loro posizione nella gerarchia.

✅ Guida d'esame: `/memory` è un comando **diagnostico** per vedere quali file di memoria sono caricati.

⚠️ **Guida vs docs:** oggi `/memory` elenca le posizioni dei file di memoria e `/context` mostra cosa è stato caricato. In ogni caso nessuno dei due attiva nulla.

Trappola 3 — Un `CLAUDE.md` di cartella per convenzioni che attraversano molte cartelle

Perché è sbagliato: vale solo per la sua cartella.

✅ Usa **regole per percorso** in `.claude/rules/` con pattern glob (es. `**/*.test.ts`). Vedi anche trappola 9.

---

2. Comandi e skill personalizzati

Percorso	Struttura	Ambito
`.claude/skills/review/SKILL.md`	**cartella** con `SKILL.md`	progetto ✅
`.claude/commands/review.md`	**file `.md` piatto**	progetto ✅
`~/.claude/skills/…`, `~/.claude/commands/…`	come sopra	personale ❌

Entrambi i percorsi di progetto producono `/review`; `.claude/skills/` è la posizione canonica e con più funzionalità (frontmatter, file di supporto, `context: fork`…).

Trappola 4 — Creare `.claude/skills/review.md` (file piatto) e aspettarsi `/review`

Perché è sbagliato: una skill è una **cartella** con un `SKILL.md` all'interno. Un file sciolto in `.claude/skills/` non viene rilevato.

✅ `.claude/skills/review/SKILL.md`, oppure il file piatto in `.claude/commands/review.md`.

Trappola 5 — Mettere un comando del team in `~/.claude/commands/` o `~/.claude/skills/`

Perché è sbagliato: sono percorsi personali, non versionati.

✅ `.claude/skills/` (preferibile) o `.claude/commands/` nel repository.

Trappola 6 — Pensare che le skill funzionino come `CLAUDE.md` (guida sempre attiva)

Perché è sbagliato: le skill si caricano **su richiesta**, come workflow per un compito; non condizionano ogni sessione.

✅ Convenzioni da applicare a ogni modifica → `CLAUDE.md` o `.claude/rules/`.

Trappola 7 — Eseguire inline nella conversazione principale una skill di analisi molto verbosa

Perché è sbagliato: tutto l'output finisce nel contesto principale, consuma token e peggiora le risposte successive.

✅ `context: fork` nel frontmatter della skill: gira in un **sottoagente isolato** e restituisce solo il risultato.

Trappola 8 — Mettere workflow specifici in `CLAUDE.md`

Perché è sbagliato: `CLAUDE.md` è sempre caricato: va riservato a standard universali.

✅ Procedure per compiti specifici (code review, routine di analisi, template di brainstorming) → **skill** invocate quando servono.

---

3. Regole per percorso (caricamento condizionale)

Esempio di regola:

```
---
paths:
  - "**/*.test.ts"
  - "infra/**/*.tf"
---
Usa describe/it, niente snapshot test...
```

Trappola 9 — `CLAUDE.md` di cartella invece di regole per percorso per convenzioni sparse

Perché è sbagliato: con file di test sparsi in 50+ cartelle servirebbe un `CLAUDE.md` in ognuna: manutenzione enorme.

✅ Una regola in `.claude/rules/` con glob.

Trappola 10 — Convenzioni specifiche per tipo di file nel `CLAUDE.md` radice

Perché è sbagliato: il `CLAUDE.md` radice si carica sempre: le convenzioni Terraform occupano token anche mentre modifichi componenti React.

✅ Regole per percorso: si caricano solo quando si lavora su file corrispondenti.

Trappola 11 — Confondere skill e regole per percorso per l'applicazione automatica delle convenzioni

Perché è sbagliato: entrambe possono attivarsi in base ai percorsi, ma:

- una **regola**, una volta letto un file corrispondente, resta nel contesto come guida e influenza **ogni modifica**;
- una **skill** è un workflow per un compito, caricato su richiesta.

✅ Convenzioni automatiche per tipo di file → **regole per percorso**.

---

4. Plan mode vs esecuzione diretta

Situazione	Scelta
Cambio architetturale, molti file, più approcci validi	**Plan mode** dall'inizio
Bug in un solo file, stack trace chiara, causa nota	**Esecuzione diretta**
Esplorazione verbosa del codebase	**Sottoagente Explore**

Trappola 12 — Esecuzione diretta per modifiche architetturali multi-file

Perché è sbagliato: le dipendenze scoperte tardi costringono a rifare lavoro costoso.

✅ **Plan mode** quando ci sono decisioni architetturali o molti file coinvolti.

Trappola 13 — Plan mode per un fix su un solo file con stack trace chiara

Perché è sbagliato: problema, posizione e soluzione sono già noti: la pianificazione è solo overhead.

✅ Esecuzione diretta.

Trappola 14 — Fare un'esplorazione molto dettagliata del codebase nella conversazione principale

Perché è sbagliato: elenchi di file ed estratti di codice riempiono il contesto e peggiorano le risposte successive.

✅ Il **sottoagente Explore** esplora in isolamento e restituisce un riassunto: il contesto principale resta pulito per l'implementazione.

Trappola 15 — Partire in esecuzione diretta e passare al plan mode "se emerge complessità"

Perché è sbagliato: se la complessità è già scritta nei requisiti (es. "spezzare il monolite in microservizi") non è un'ipotesi, è un dato.

✅ Plan mode **fin dall'inizio**.

---

5. Tecniche di raffinamento iterativo

Problema	**Tecnica**
Trasformazione nota interpretata in modo incoerente	**2-3 esempi input/output**
Requisiti poco chiari, considerazioni non previste	**Pattern "intervista"** (Claude fa domande)
Più correzioni che interagiscono tra loro	**Un unico messaggio** con tutte
Correzioni indipendenti	In sequenza

Trappola 16 — Raffinare la descrizione in prosa quando il modello la interpreta in modo incoerente

Perché è sbagliato: una prosa più precisa richiede ancora interpretazione: la deriva continua.

✅ **2-3 esempi concreti di input/output** mostrano direttamente il pattern.

Trappola 17 — Inviare correzioni che interagiscono un messaggio alla volta

Perché è sbagliato: ogni correzione viene applicata a uno stato che la successiva cambia; si rischia di disfare il lavoro precedente.

✅ Correzioni interdipendenti → **un unico messaggio dettagliato**; quelle indipendenti → in sequenza.

Trappola 18 — Pattern "intervista" quando lo sviluppatore conosce già esattamente la trasformazione

Perché è sbagliato: l'intervista serve a far emergere considerazioni non previste; qui non ce ne sono.

✅ Esempi input/output.

---

6. Integrazione CI/CD

Trappola 19 — `CLAUDE_HEADLESS=true`, `--batch` o redirezione di stdin per un job CI bloccato in attesa di input

Perché è sbagliato: `CLAUDE_HEADLESS` e `--batch` **non esistono** (distrattori inventati); la redirezione di stdin è un trucco Unix che non usa la sintassi di Claude Code.

✅ **`claude -p "prompt"`** (modalità print/non interattiva): esegue, scrive su stdout ed esce. Utili in CI anche `--output-format json` e `--allowedTools`.

🔎 Flag dal nome plausibile ma mai visto nella documentazione → probabilmente inventato.

Trappola 20 — Considerare l'auto-revisione nella stessa sessione efficace quanto una revisione indipendente

Perché è sbagliato: la sessione conserva il ragionamento con cui il codice è stato generato e tende a confermarlo.

✅ Un'**istanza separata**, senza quel contesto, trova più problemi.

Trappola 21 — Batch API per i controlli CI pre-merge

Perché è sbagliato: la Message Batches API può impiegare fino a 24 ore e non ha SLA di latenza; un check pre-merge blocca lo sviluppatore.

✅ Controlli bloccanti → **API sincrona**; analisi notturne o settimanali → Batch (circa 50% di risparmio).

Trappola 22 — Per evitare commenti ripetuti, rivedere solo il diff dell'ultimo commit o riprendere la sessione della prima esecuzione

Perché è sbagliato:

- il diff incrementale nasconde i problemi segnalati prima e non ancora risolti;
- la sessione ripresa contiene risultati degli strumenti obsoleti su codice ormai cambiato.

✅ **Nuova revisione completa della PR**, passando i risultati precedenti e chiedendo di segnalare solo problemi **nuovi o non ancora risolti**.